

Towards a Decentralized Sphinx Header Construction

Aurélien Chassagne

August 25, 2025

① Introduction

- Motivation - Speak about importance of Privacy on the network
- Some Technologies: TOR vs Mixnet (lot of animations)

② The problem

- Mixnet's problem (discard illegal packet at exit gateway)
- Our idea - Decentralization

③ Sphinx

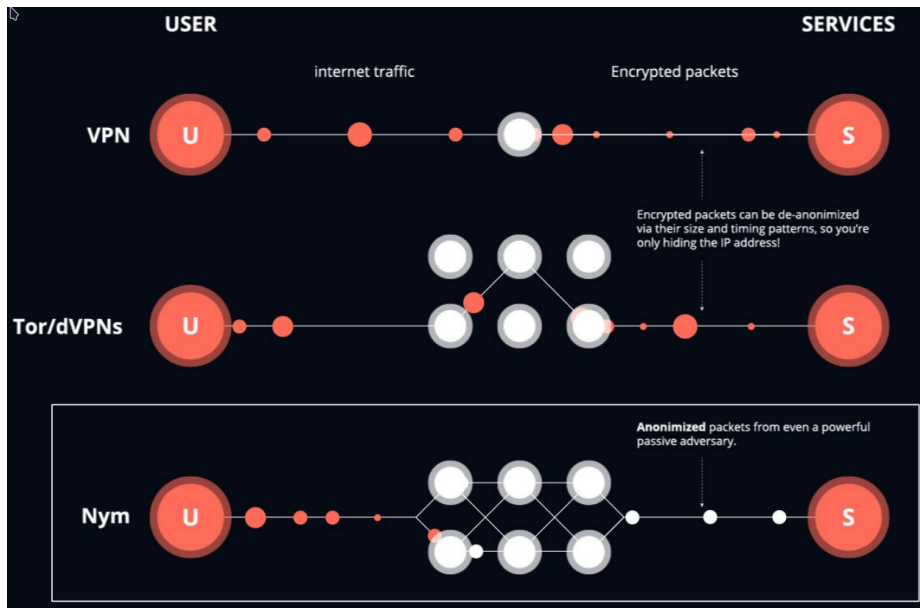
- Original Sphinx Processing (lot of animations)
- Original Sphinx Construction (lot of animations) (in appendix for conference presentation)
- ECC slide (in appendix for conference presentation)
- Our Sphinx implementation (lot of animations)

④ Tests and Results

⑤ Conclusion and future work

– Why privacy matters –

Network Privacy Technologies



The Problem - Policy enforcement at exit gateway

Why *policy enforcement*:

- ① **Economic:** Company willing to provide different subscription plans
- ② **Political:** Enforce law regulation (i.e. reaching illegal website)

Why at *exit gateway*:

- ① Privacy-friendly legitimacy verification of layered encrypted packet is tricky
- ② At exit gateway, the destination is revealed allowing easier policy enforcement

Why is it a *problem*:

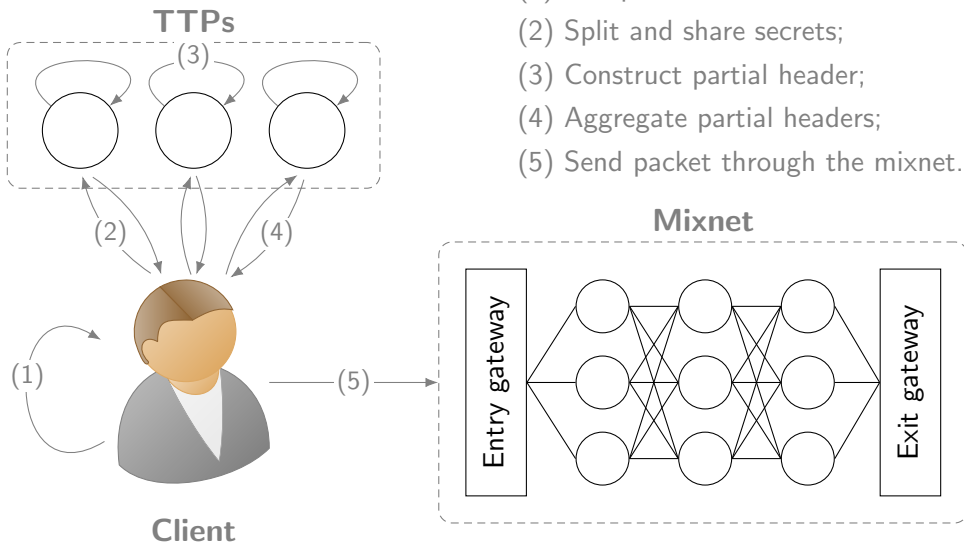
- ① Computation waste of mixnodes

What we would like:

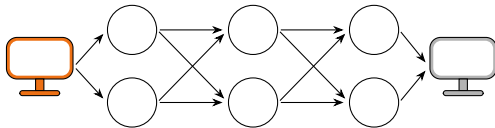
- Policy enforcement applicable at any nodes

Our Idea - Decentralized packet construction

- (1) Compute route & shared secrets;
- (2) Split and share secrets;
- (3) Construct partial header;
- (4) Aggregate partial headers;
- (5) Send packet through the mixnet.

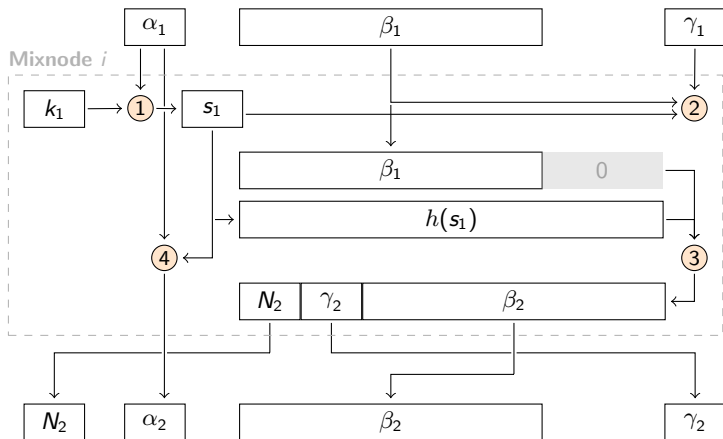
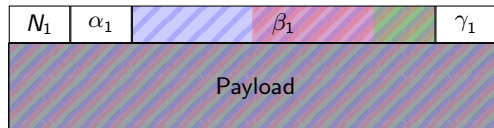


Original Sphinx - Decryption

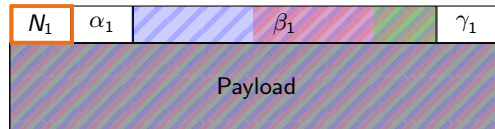
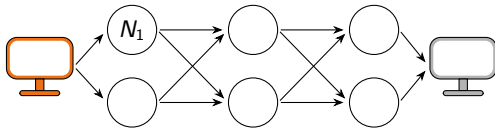


Steps:

- 1 Compute share secret
- 2 Integrity verification
- 3 Decrypt Header
- 4 Update cryptographics
- 5 Decrypt Payload

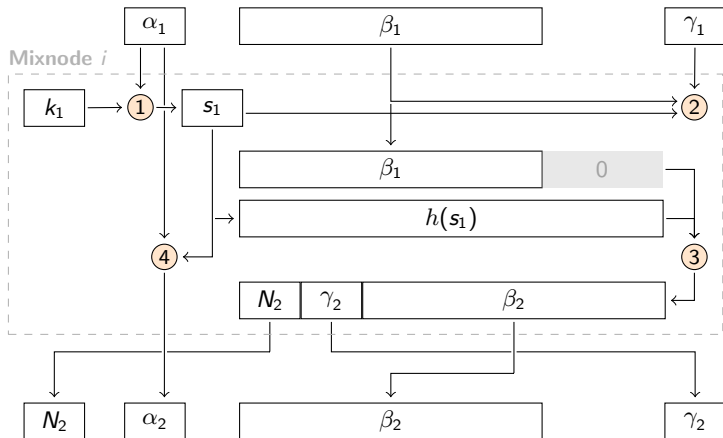


Original Sphinx - Decryption

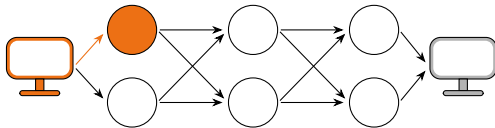


Steps:

- 1 Compute share secret
- 2 Integrity verification
- 3 Decrypt Header
- 4 Update cryptographics
- 5 Decrypt Payload

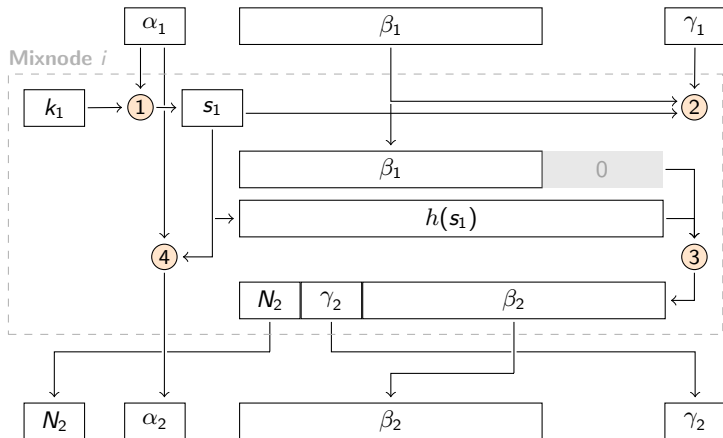
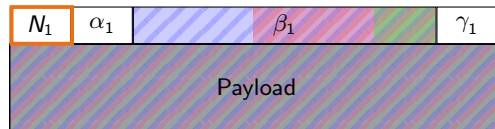


Original Sphinx - Decryption

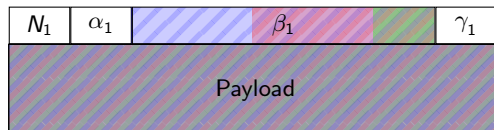
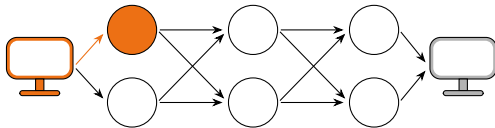


Steps:

- 1 Compute share secret
- 2 Integrity verification
- 3 Decrypt Header
- 4 Update cryptographics
- 5 Decrypt Payload

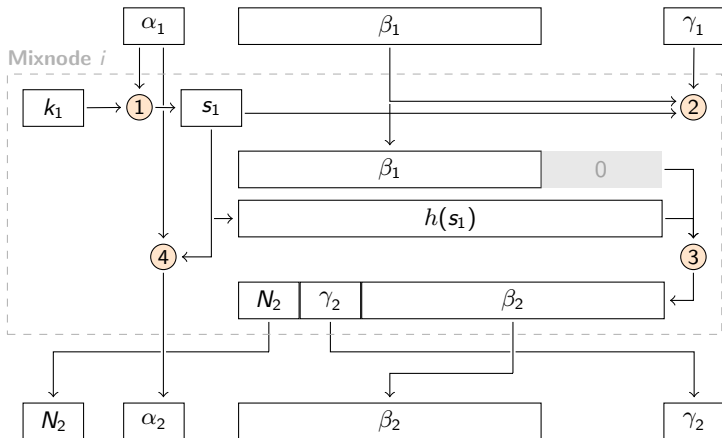


Original Sphinx - Decryption

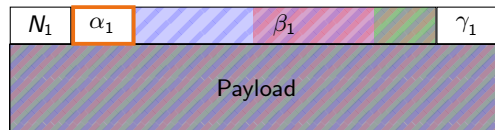
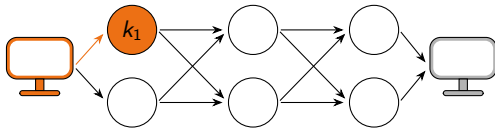


Steps:

- 1 Compute share secret
- 2 Integrity verification
- 3 Decrypt Header
- 4 Update cryptographics
- 5 Decrypt Payload

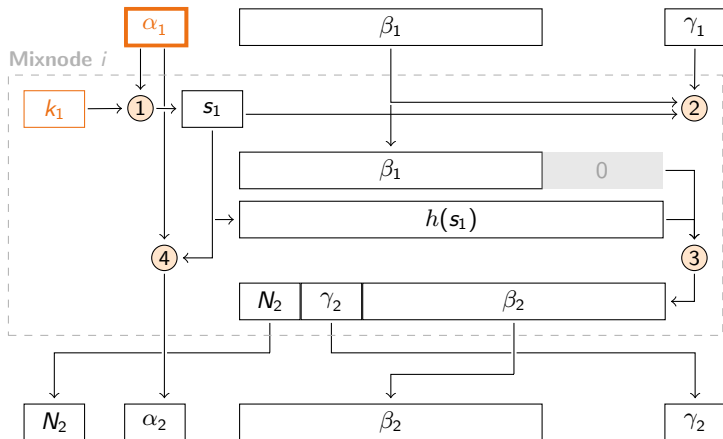


Original Sphinx - Decryption

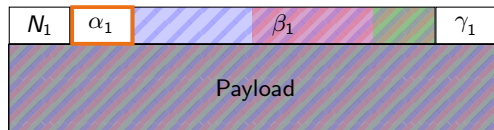
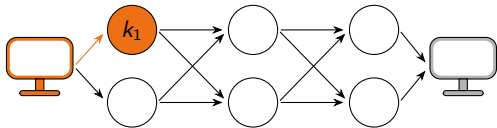


Steps:

- 1 Compute share secret
- 2 Integrity verification
- 3 Decrypt Header
- 4 Update cryptographics
- 5 Decrypt Payload

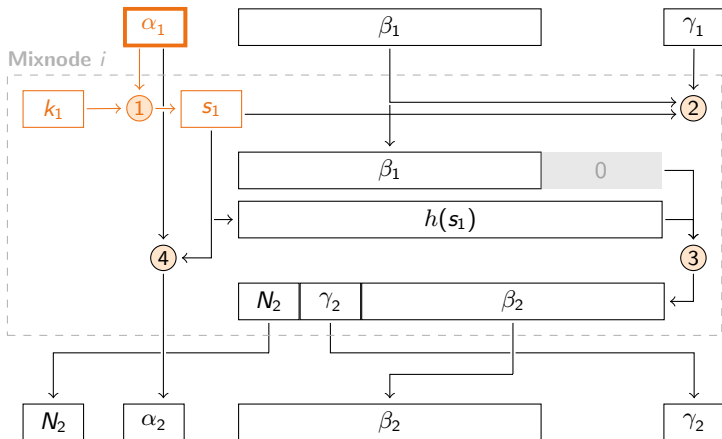


Original Sphinx - Decryption

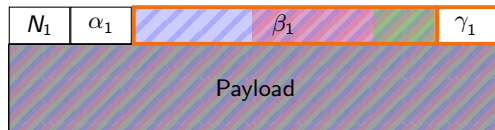
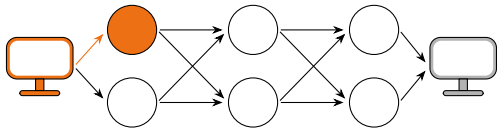


Steps:

- 1 Compute share secret
- 2 Integrity verification
- 3 Decrypt Header
- 4 Update cryptographics
- 5 Decrypt Payload

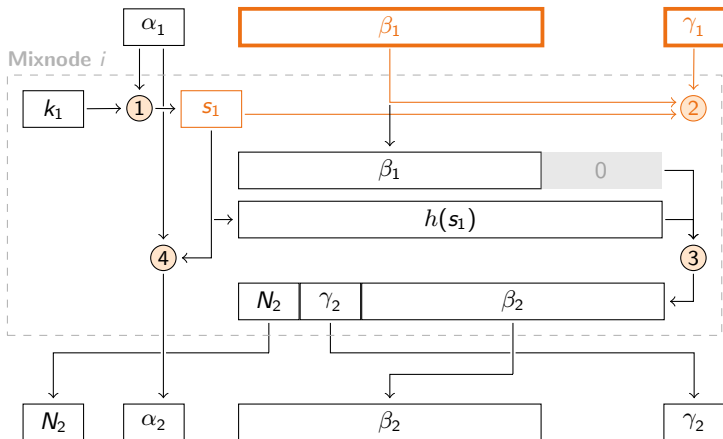


Original Sphinx - Decryption

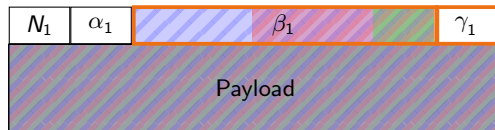
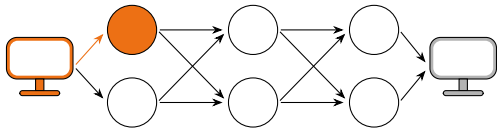


Steps:

- 1 Compute share secret
- 2 Integrity verification
- 3 Decrypt Header
- 4 Update cryptographics
- 5 Decrypt Payload

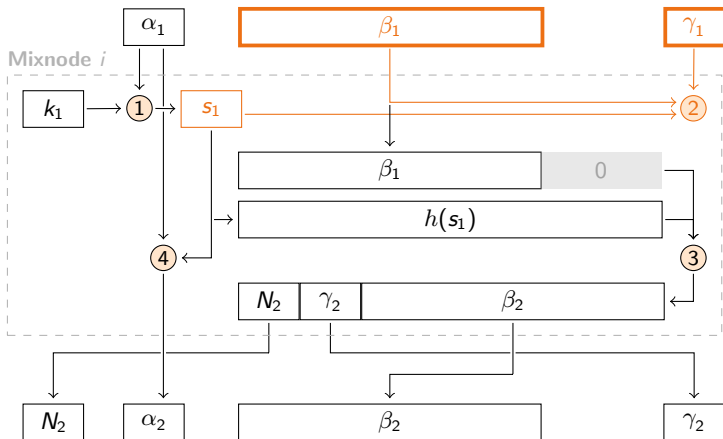


Original Sphinx - Decryption

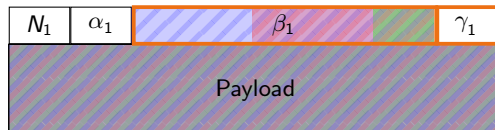
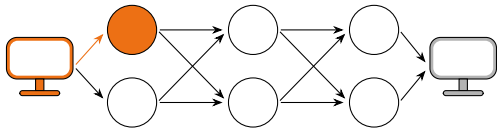


Steps:

- 1 Compute share secret
- 2 Integrity verification
- 3 Decrypt Header
- 4 Update cryptographics
- 5 Decrypt Payload

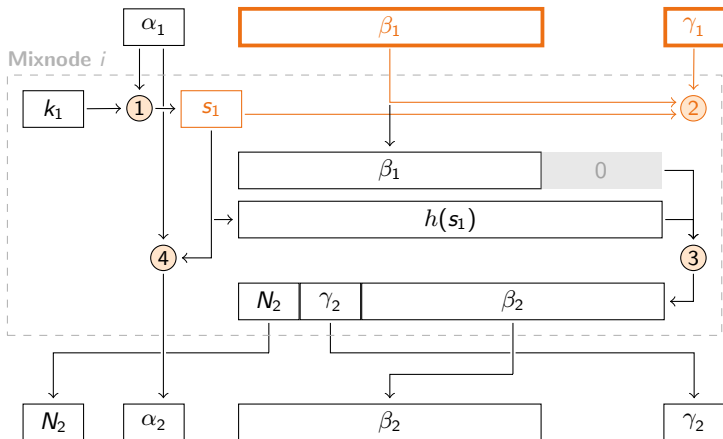


Original Sphinx - Decryption



Steps:

- 1 Compute share secret
- 2 Integrity verification
- 3 Decrypt Header
- 4 Update cryptographics
- 5 Decrypt Payload



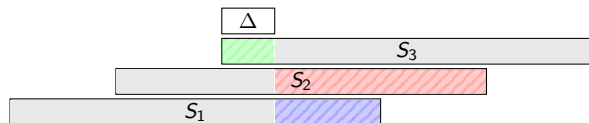
Original Sphinx - Encryption



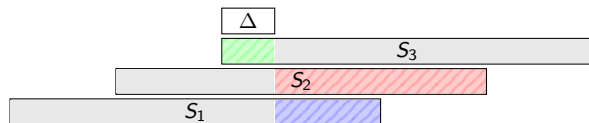
Original Sphinx - Encryption



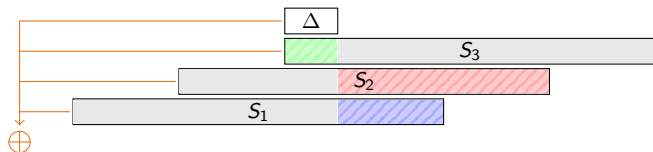
Original Sphinx - Encryption



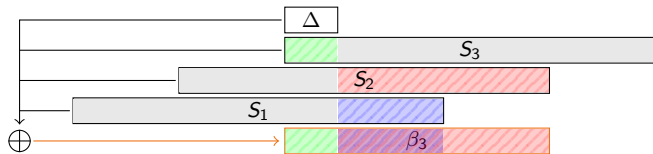
Original Sphinx - Encryption



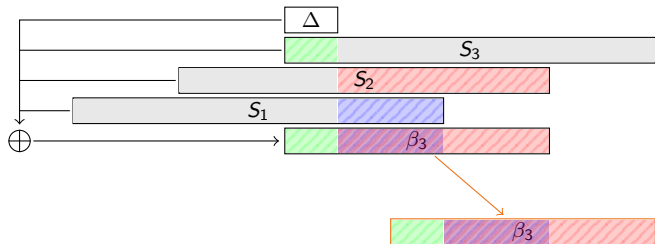
Original Sphinx - Encryption



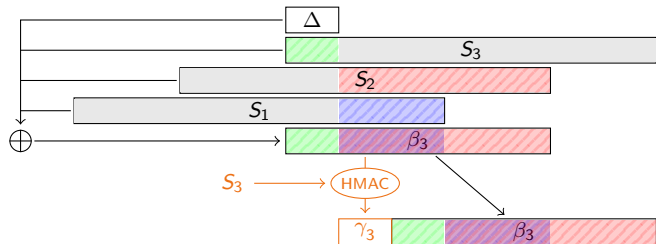
Original Sphinx - Encryption



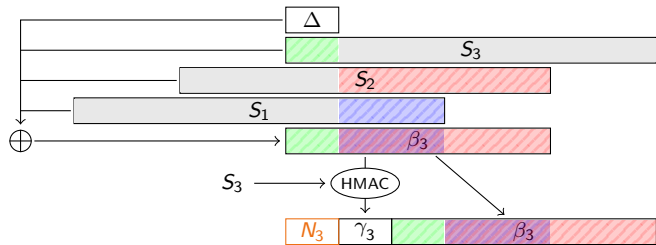
Original Sphinx - Encryption



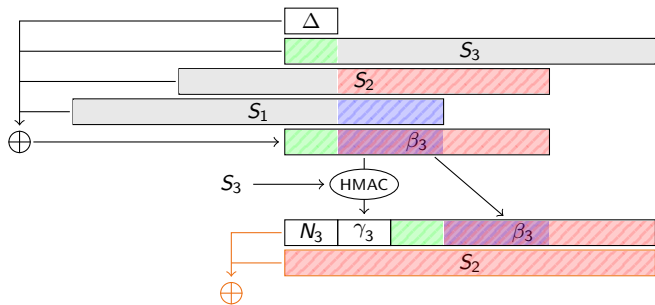
Original Sphinx - Encryption



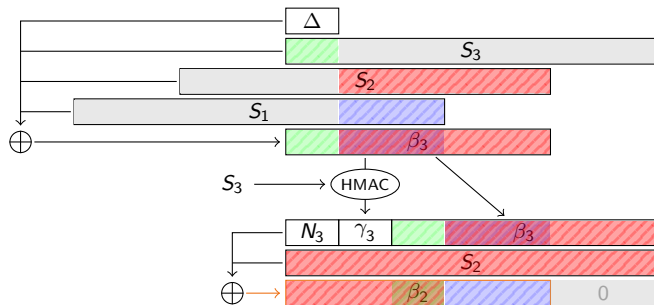
Original Sphinx - Encryption



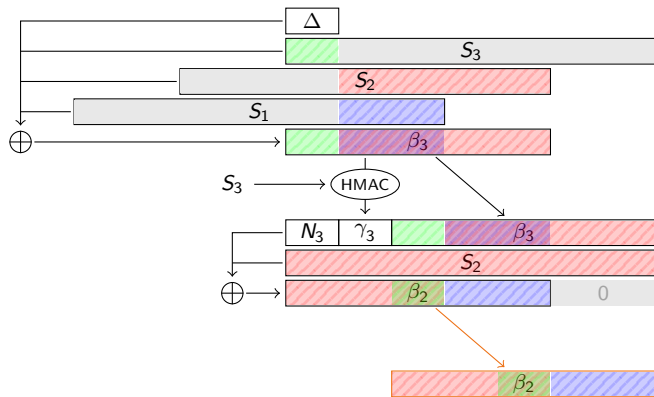
Original Sphinx - Encryption



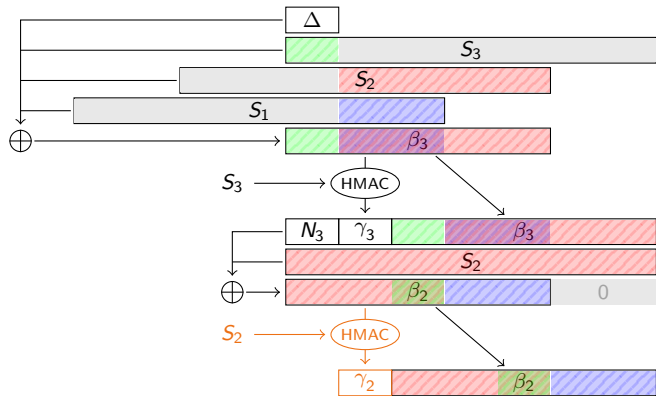
Original Sphinx - Encryption



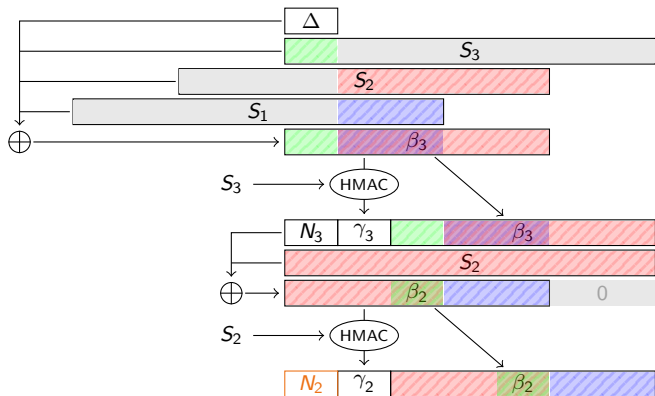
Original Sphinx - Encryption



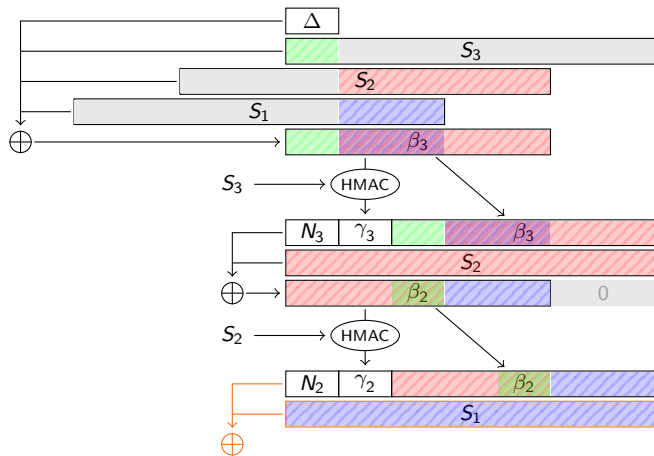
Original Sphinx - Encryption



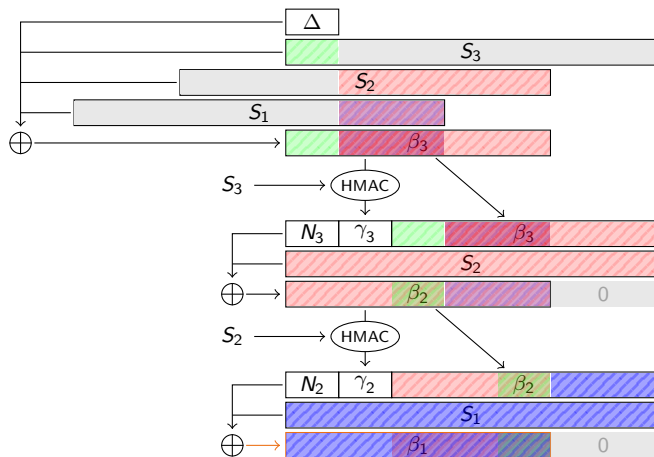
Original Sphinx - Encryption



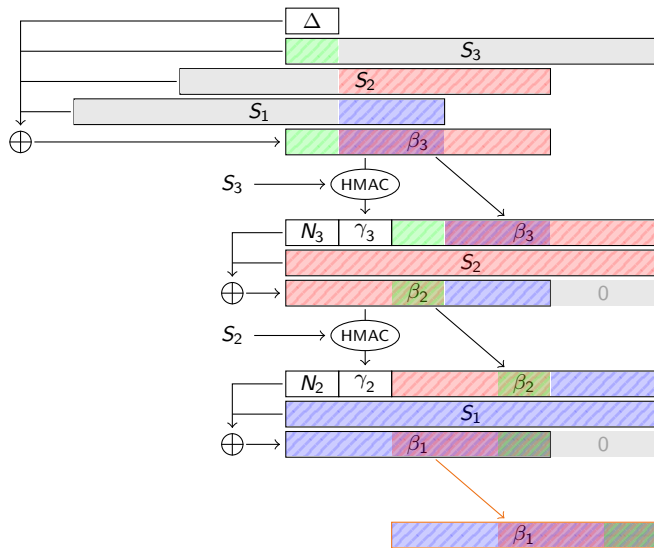
Original Sphinx - Encryption



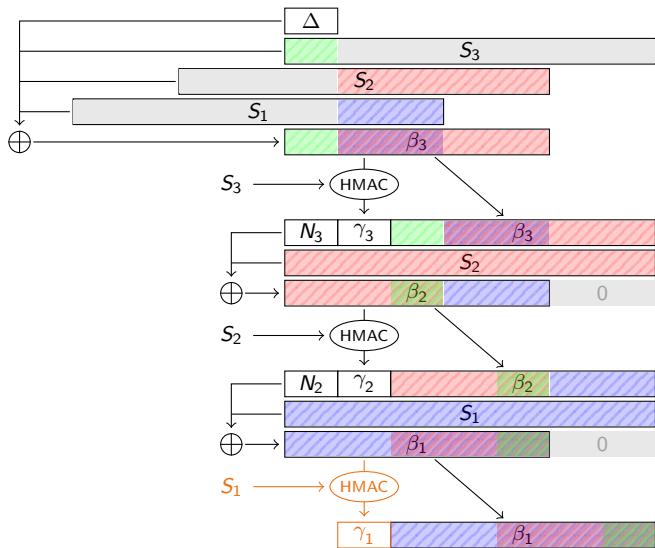
Original Sphinx - Encryption



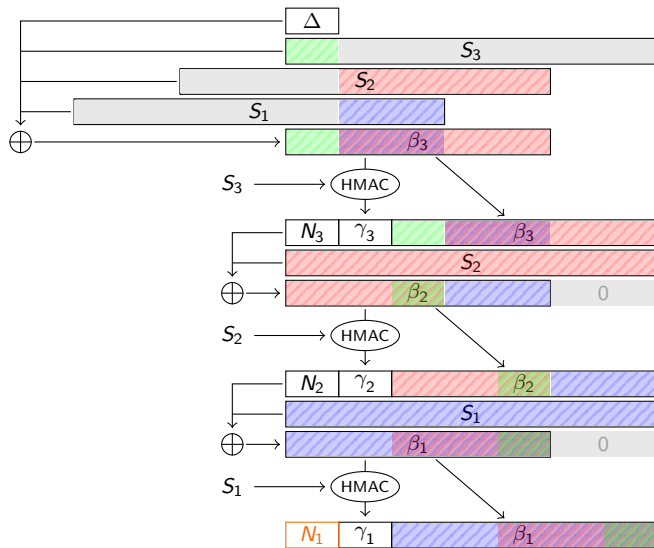
Original Sphinx - Encryption



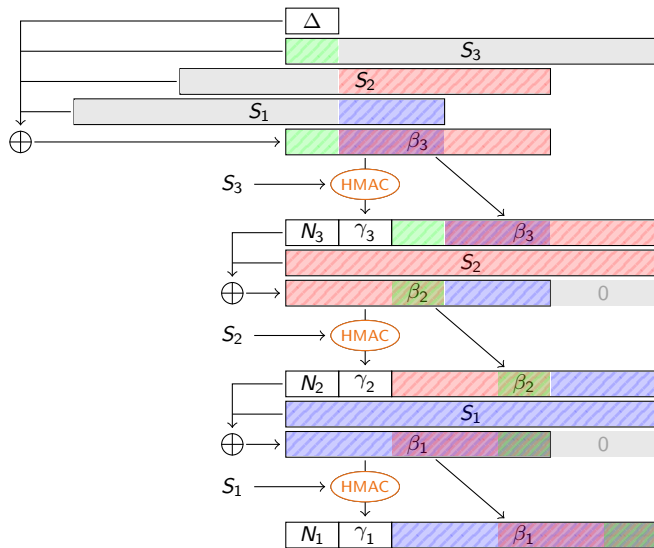
Original Sphinx - Encryption



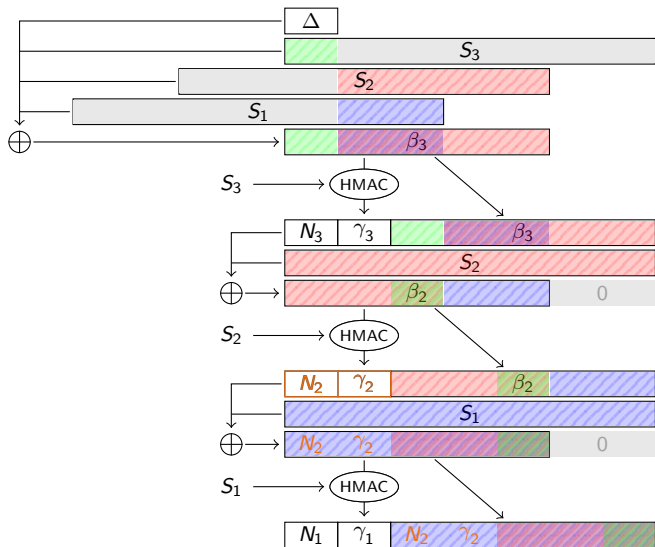
Original Sphinx - Encryption



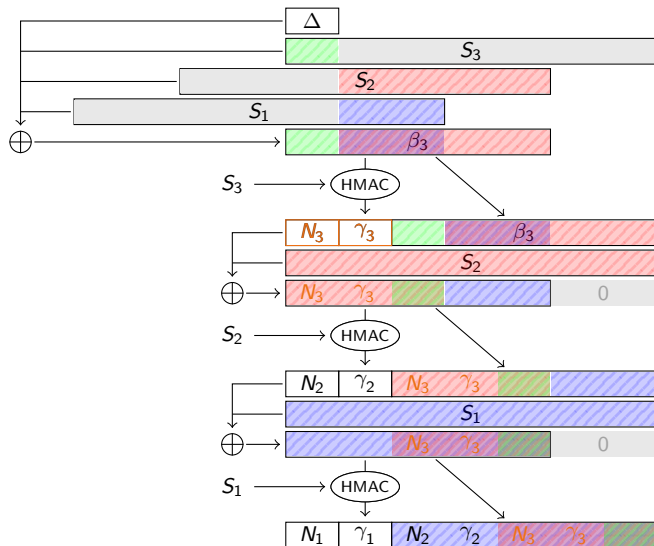
Original Sphinx - Encryption



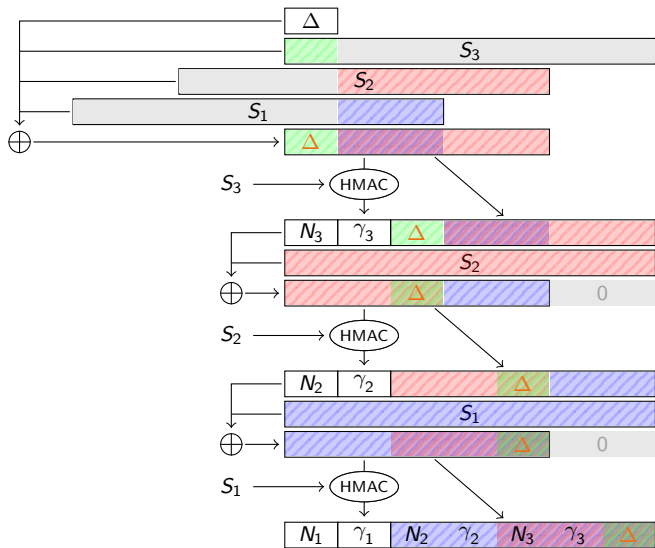
Original Sphinx - Encryption



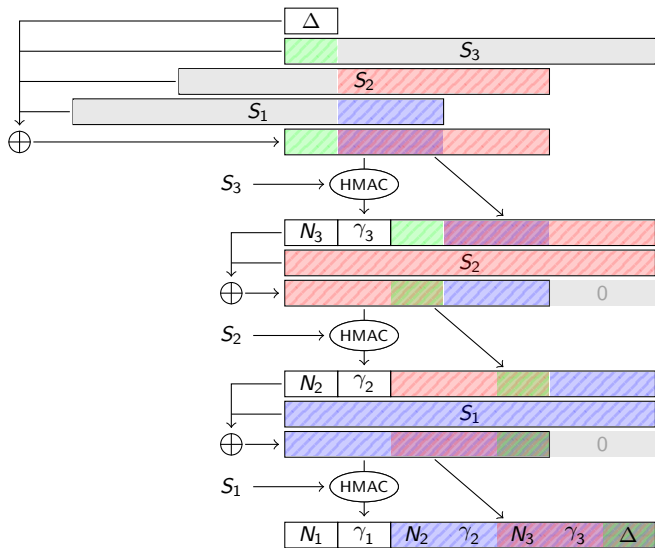
Original Sphinx - Encryption



Original Sphinx - Encryption



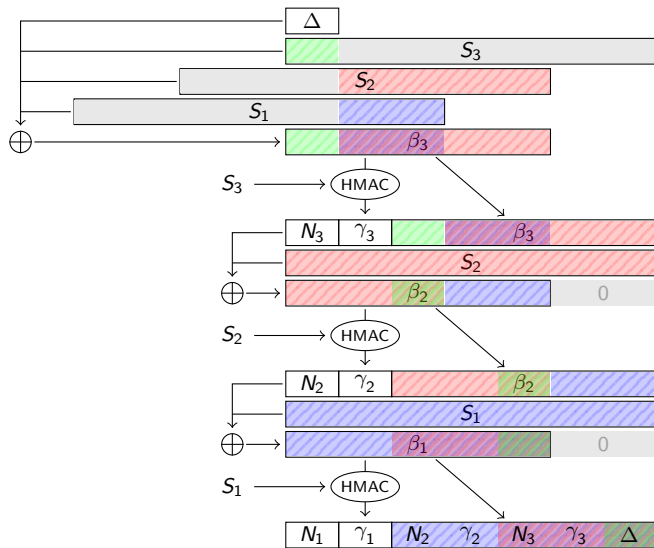
Original Sphinx - Encryption



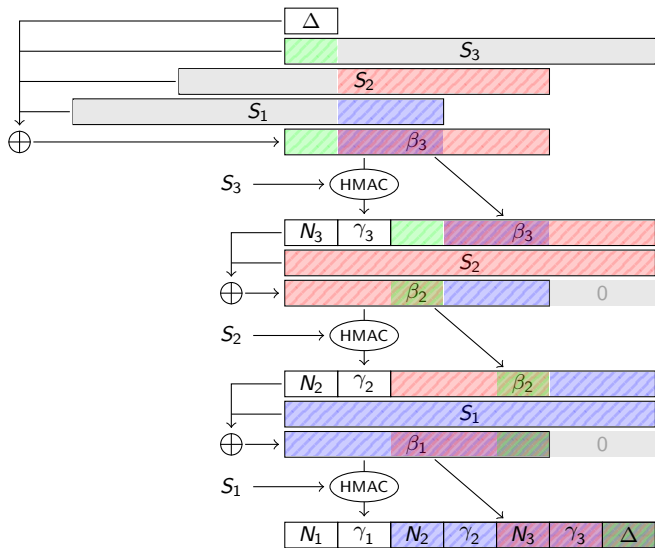
ECC slide

Is it needed ? Generator ?

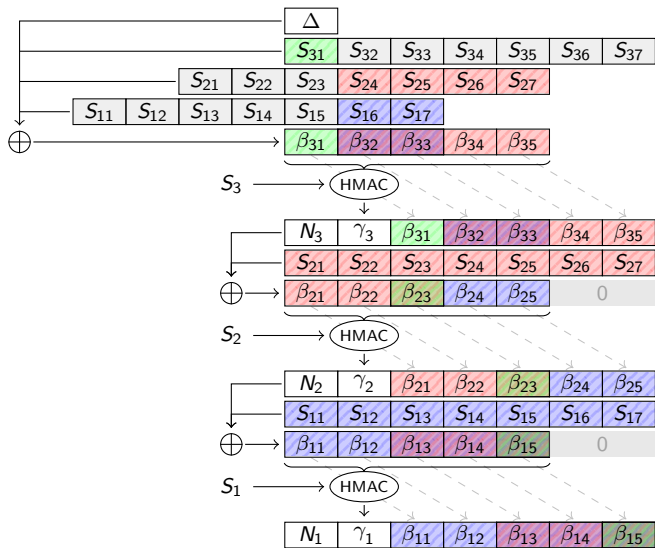
Our decentralized Sphinx construction



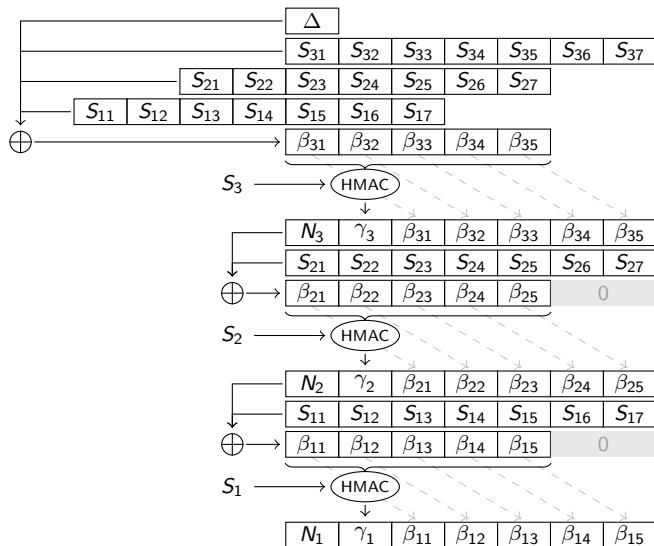
Our decentralized Sphinx construction



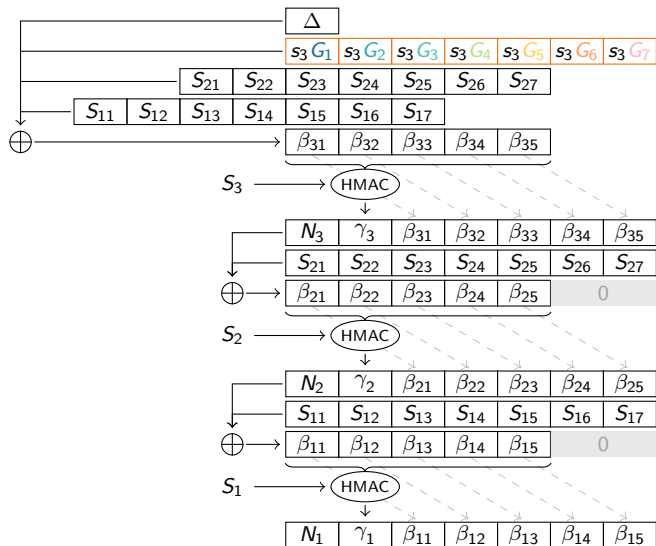
Our decentralized Sphinx construction



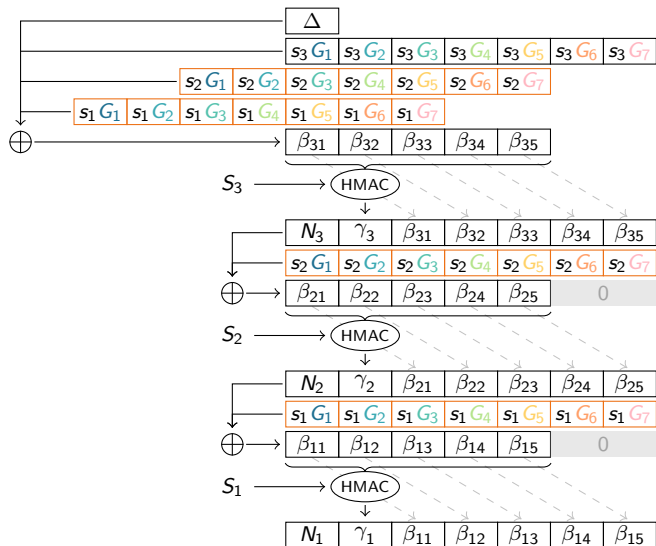
Our decentralized Sphinx construction



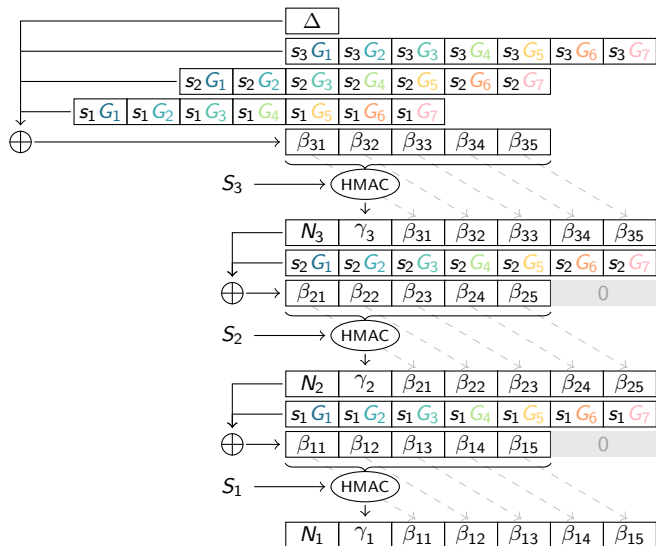
Our decentralized Sphinx construction



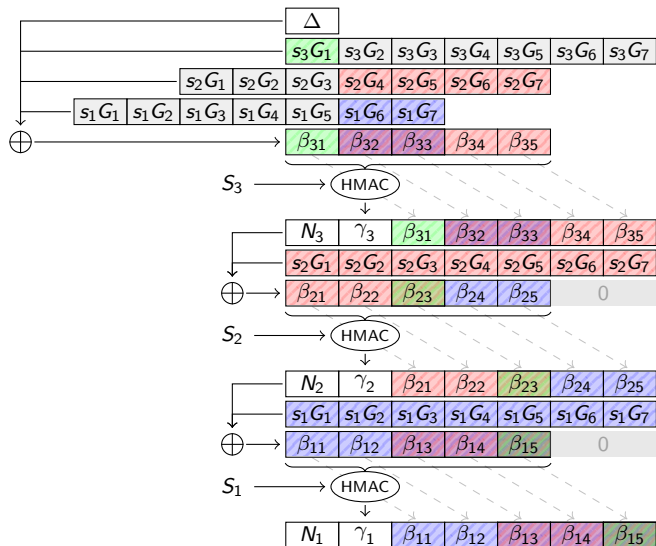
Our decentralized Sphinx construction



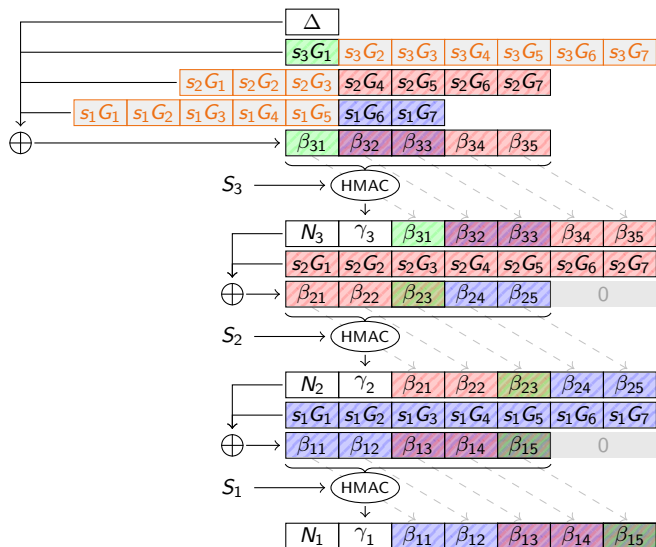
Our decentralized Sphinx construction



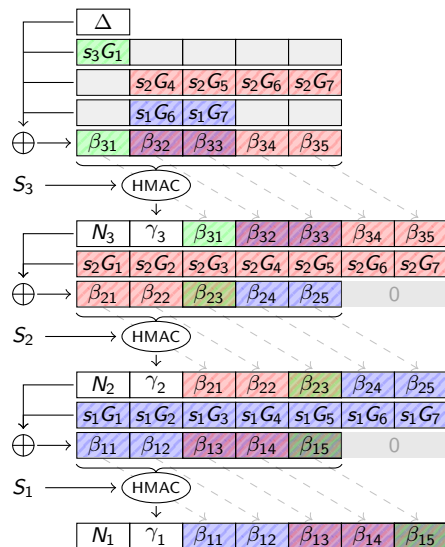
Our decentralized Sphinx construction



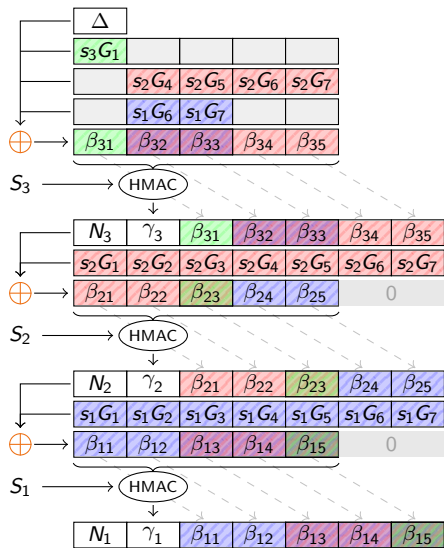
Our decentralized Sphinx construction



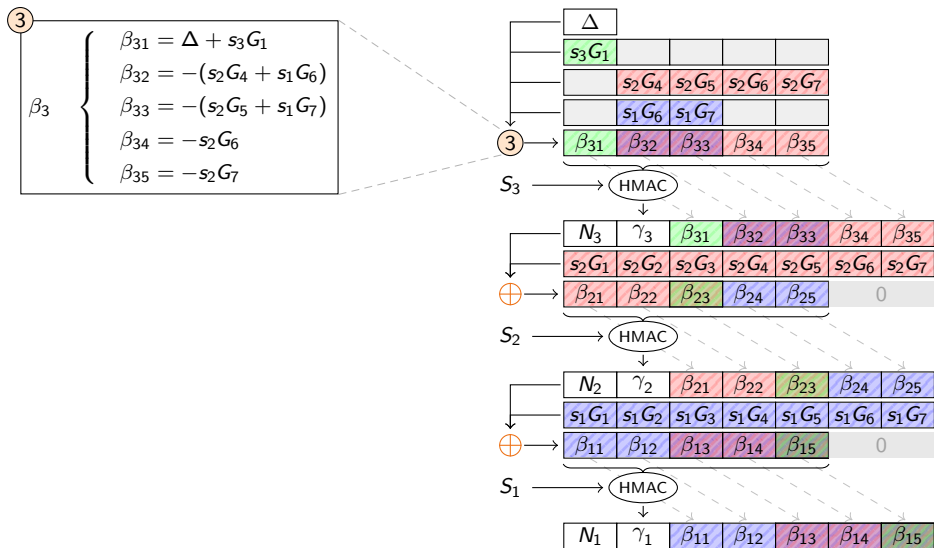
Our decentralized Sphinx construction



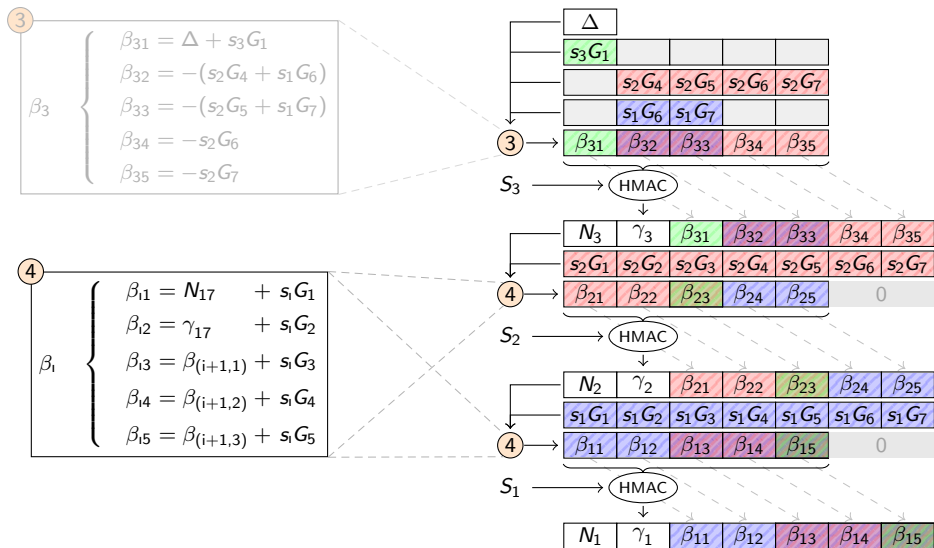
Our decentralized Sphinx construction



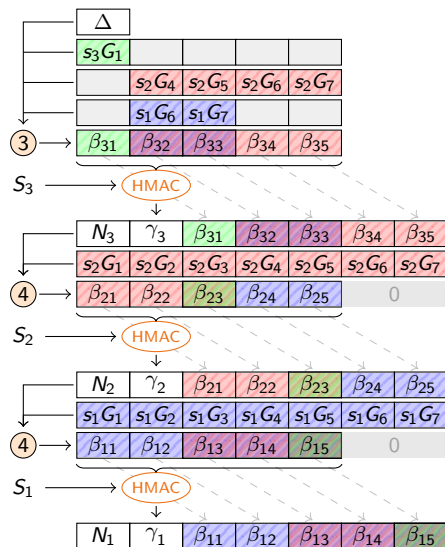
Our decentralized Sphinx construction



Our decentralized Sphinx construction



Our decentralized Sphinx construction



Our decentralized Sphinx construction

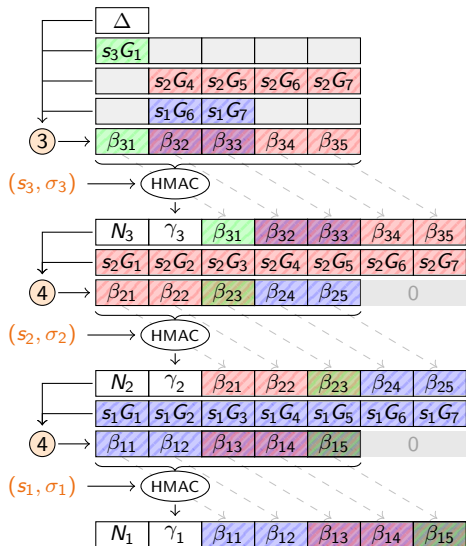


$$s_1 \neq h(\sigma_1)$$



σ_i : **hash** of shared secret with mixnode i

s_i : **share** of shared secret with mixnode i



Our decentralized Sphinx construction



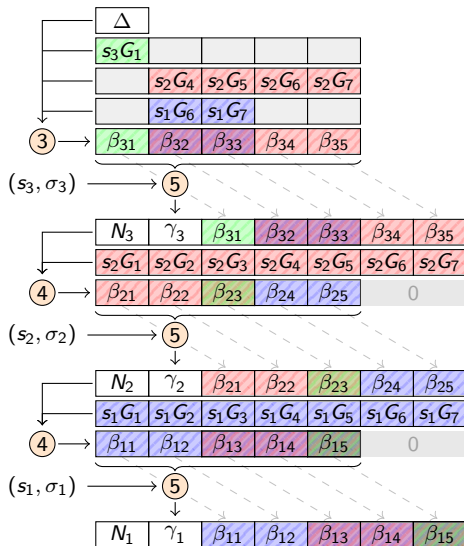
$$s_i \neq h(\sigma_i)$$



σ_i : **hash** of shared secret with mixnode i

s_i : **share** of shared secret with mixnode i

$$\gamma_i = s_i G + \sum_{j=1}^5 (h^{(j)}(\sigma_i) \beta_{ij})$$

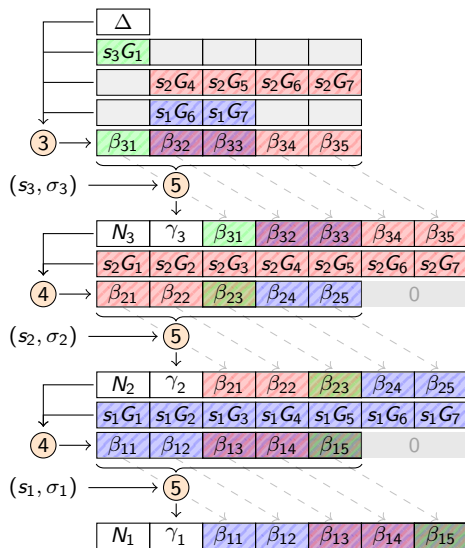


Our decentralized Sphinx construction

$$\textcircled{3} \quad \beta_3 \quad \left\{ \begin{array}{l} \beta_{31} = \Delta + s_3 G_1 \\ \beta_{32} = -(s_2 G_4 + s_1 G_6) \\ \beta_{33} = -(s_2 G_5 + s_1 G_7) \\ \beta_{34} = -s_2 G_6 \\ \beta_{35} = -s_2 G_7 \end{array} \right.$$

$$\textcircled{4} \quad \beta_i \quad \left\{ \begin{array}{l} \beta_{i1} = N_{i+1} + s_i G_1 \\ \beta_{i2} = \gamma_{i+1} + s_i G_2 \\ \beta_{i3} = \beta_{(i+1,1)} + s_i G_3 \\ \beta_{i4} = \beta_{(i+1,2)} + s_i G_4 \\ \beta_{i5} = \beta_{(i+1,3)} + s_i G_5 \end{array} \right.$$

$$\textcircled{5} \quad \gamma_i = s_i G + \sum_{j=1}^5 (h^{(j)}(\sigma_i) \beta_{ij})$$



- Complexity - Big O notation
- Unlinkability
 - NIST - P-value distribution (Fig 6 → Fig 7)
 - NIST - Proportion of success (Fig 5)
- Recap table (Tab. 2)

Conclusion and Future Work